Name: M . Sivateja

Hall Ticket: 2303A52145

Batch: 41

Task 1: Auto-Genera ng Func on Documenta on in a Shared Codebase

Python script containing multiple functions without any docstrings.

Prompt:

#generate python code for calculator that can perform basic arithmetic operations (addition, subtraction, multiplication, division based on user input. The calculator should continue to prompt the user for operations un l they choose to exit. should contain multiple functions for each opera on and a main function to run the calculator. include error handling for invalid inputs and division by zero. without any docstrings or comments.

Code:

```
def add(num1, num2):
    return num1 + num2

def subtract(num1, num2):
    return num1 - num2

def multiply(num1, num2):
    return num1 * num2

def divide(num1, num2):
    if num2 == 0:
        return 'Error: Division by zero is not allowed.'
    return num1 / num2

def get_number_input(prompt):
    while True:
        try:
            user_input = input(prompt)
            number = float(user_input)
            return number
        except ValueError:
            print('Invalid input. Please enter a number.')

def calculator():
    print('\nWelcome to the Python Calculator!')
    while True:
        print('\nPlease choose an operation:')
        print('1. Addition (+)')
        print('2. Subtraction (-)')
        print('3. Multiplication (*)')
        print('4. Division (/)')
        print('5. Exit')
```

```python
        choice = input('Enter your choice (1-5): ')

        if choice == '5':
            print('Exiting calculator. Goodbye!')
            break
        elif choice in ('1', '2', '3', '4'):
            num1 = get_number_input('Enter the first number: ')
            num2 = get_number_input('Enter the second number: ')

            result = None
            if choice == '1':
                result = add(num1, num2)
            elif choice == '2':
                result = subtract(num1, num2)
            elif choice == '3':
                result = multiply(num1, num2)
            elif choice == '4':
                result = divide(num1, num2)

            if result is not None:
                print(f'Result: {result}')
        else:
            print('Invalid choice. Please enter a number between 1 and 5.')

if __name__ == '__main__':
    calculator()
```

Output:

```
Welcome to the Python Calculator!

Please choose an operation:
1. Addition (+)
2. Subtraction (-)
3. Multiplication (*)
4. Division (/)
5. Exit
Enter your choice (1-5): 1
Enter the first number: 23
Enter the second number: 43
Result: 66.0

Please choose an operation:
1. Addition (+)
2. Subtraction (-)
3. Multiplication (*)
4. Division (/)
5. Exit
Enter your choice (1-5): 5
Exiting calculator. Goodbye!
```

Exp;anation:

The code creates a simple menu-driven calculator that lets the user choose addition, subtraction, multiplication, or division. Each operation is handled by a separate function, and input validation is included to manage invalid entries and prevent errors like division by zero.

Prompt:

Write a Python program for bubble sort. should take user input for the list of numbers to sort.

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

try:
    user_input = input("Enter numbers separated by spaces: ")
    numbers = [float(x) for x in user_input.split()]
    print("Original list:", numbers)
    sorted_list = bubble_sort(numbers)
    print("Sorted list:", sorted_list)
except ValueError:
    print("Invalid input. Please enter numerical values.")
```

Explanation:

AI assistance is used to insert inline comments only in parts of the code where the logic is complex or unclear, while avoiding comments for obvious sections. This improves readability without cluttering the code with unnecessary explanations.

Task 3: Genera ng Module-Level Documenta on for a Python Package

Prompt:

Generate a professional module-level docstring for this Python file.The docstring should explain the purpose of the module, required dependencies, describe the key classes and funcitons, and include a short example of how the module can be used.

Code:

```
"""
Bubble Sort Module

This module implements the Bubble Sort algorithm, a straightforward comparison-
based
sorting technique. It includes a function to sort numerical arrays in-place and
a command-line interface for user interaction.

Dependencies:
    None (Uses only Python built-in features).

Functions:
    bubble_sort(arr):
        Sorts a list of numbers in ascending order using the bubble sort algorithm.
```

```
Example Usage:
    >>> data = [64, 34, 25, 12, 22]
    >>> sorted_data = bubble_sort(data)
    >>> print(sorted_data)
    [12, 22, 25, 34, 64]
"""

def bubble_sort(arr):
    """Sorts a list of numbers in-place and returns the sorted list."""
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

if __name__ == "__main__":
    try:
        user_input = input("Enter numbers separated by spaces: ")
        numbers = [float(x) for x in user_input.split()]
        print("Original list:", numbers)
        sorted_list = bubble_sort(numbers)
        print("Sorted list:", sorted_list)
    except ValueError:
        print("Invalid input. Please enter numerical values separated by spaces.")
```

Code:

```
•••  Enter numbers separated by spaces: 5 6
     Original list: [5.0, 6.0]
     Sorted list: [5.0, 6.0]
```

Explanation:

This task shows how to create a simple Python module and then use an AI tool to produce clear, well-structured, and professional documentation for it

Task 4: Conver ng Developer Comments into Structured Docstrings :

Prompt:

Convert the inline comments inside each func on into proper Google-style docstrings. Preserve the original meaning, remove redundant inline comments, and keep the func on logic unchanged.

```
def bubble_sort(arr):
    """Sorts a list of numbers using the bubble sort algorithm.

    Args:
        arr (list): A list of numerical values to be sorted.

    Returns:
        list: The sorted list (sorted in-place).
    """
    n = len(arr)
```

```
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

if __name__ == "__main__":
    try:
        user_input = input("Enter numbers separated by spaces: ")
        numbers = [float(x) for x in user_input.split()]
        print("Original list:", numbers)
        sorted_list = bubble_sort(numbers)
        print("Sorted list:", sorted_list)
    except ValueError:
        print("Invalid input. Please enter numerical values.")
```

Output:

```
...  Enter numbers separated by spaces: 1 4 3 2
     Original list: [1.0, 4.0, 3.0, 2.0]
     Sorted list: [1.0, 2.0, 3.0, 4.0]
```

Explanation:

The code analyzes student data by calculating the average marks and determining the highest-performing student. Instead of using inline comments, it uses structured docstrings to make the documentation clearer, more organized, and consistent.

Task 5: **Building a Mini Automa c Documenta on Generator**

Prompt:

Scan the file calculator.py and add placeholder Google-style docstrings for every class and function that does not already have one. Do not change any existing logic or behavior; only insert clean, structured docstrings.

Code:

```
import ast
import astunparse

def generate_placeholder_docstrings(source_code):
    tree = ast.parse(source_code)

    class DocstringAdder(ast.NodeTransformer):
        def visit_FunctionDef(self, node):
            if not ast.get_docstring(node):
                placeholder = f"""Placeholder docstring for
{node.name}.\n\n    Args:\n        *args: Description of
arguments.\n\n    Returns:\n        Description of the return value.\n    """'
                docstring_node = ast.Expr(value=ast.Constant(value=placeholder))
                node.body.insert(0, docstring_node)
            return node
```

```
        def visit_ClassDef(self, node):
            if not ast.get_docstring(node):
                placeholder = f'"""Placeholder docstring for class {node.name}."""'
                docstring_node = ast.Expr(value=ast.Constant(value=placeholder))
                node.body.insert(0, docstring_node)
            return node

    transformer = DocstringAdder()
    modified_tree = transformer.visit(tree)
    return astunparse.unparse(modified_tree)

# Example usage with a mock calculator.py content
calculator_source = """
def add(a, b):
    return a + b

class Calculator:
    def subtract(self, a, b):
        return a - b
"""

processed_code = generate_placeholder_docstrings(calculator_source)
print("Code with inserted docstrings:")
print(processed_code)
```

Output:

```
... Code with inserted docstrings:


def add(a, b):
    '"""Placeholder docstring for add.\n\n    Args:\n        *args: Description of arguments.\n\n    Returns:\n        Description of the return value.\n    """'
    return (a + b)

class Calculator():
    '"""Placeholder docstring for class Calculator."""'

    def subtract(self, a, b):
        return (a - b)
```

Task 5: Building a Mini Automatic Documenta on Generator

Prompt:

Scan the file calculator.py and add placeholder Google-style docstrings for every class and function that does not already have one. Do not change any existing logic or behavior; only insert clean, structured docstrings.

Code:

```
import ast

class DocstringScaffoldTransformer(ast.NodeTransformer):
    """AST Transformer to insert Google-style docstring templates into classes and
functions."""

    def _has_docstring(self, node):
        if node.body and isinstance(node.body[0], ast.Expr):
            if isinstance(node.body[0].value, (ast.Constant, ast.Str)):
```

```
                val = node.body[0].value.value if isinstance(node.body[0].value,
ast.Constant) else node.body[0].value.s
                if isinstance(val, str):
                    return True
        return False

    def visit_FunctionDef(self, node):
        if not self._has_docstring(node):
            docstring_text = f'"""Placeholder docstring for
{node.name}.\n\nArgs:\n    *args: Description of
arguments.\n\nReturns:\n    Description of the return value.\n"""'
            new_docstring = ast.Expr(value=ast.Constant(value=docstring_text))
            node.body.insert(0, new_docstring)

        self.generic_visit(node)
        return node

    def visit_ClassDef(self, node):
        if not self._has_docstring(node):
            docstring_text = f'"""Placeholder docstring for class {node.name}."""'
            new_docstring = ast.Expr(value=ast.Constant(value=docstring_text))
            node.body.insert(0, new_docstring)

        self.generic_visit(node)
        return node

def generate_docstring_scaffolds(source_code):
    """Wrapper function to parse source, apply docstring transformations, and
unparse back to code."""
    # Parse the source code into an AST
    tree = ast.parse(source_code)

    # Instantiate the transformer and apply it to the tree
    transformer = DocstringScaffoldTransformer()
    modified_tree = transformer.visit(tree)

    # Fix missing locations for the newly inserted nodes
    ast.fix_missing_locations(modified_tree)

    # Convert the AST back to source code string
    return ast.unparse(modified_tree)

# --- Demonstration ---
sample_code = """
def add(a, b):
    return a + b

class Calculator:
    def subtract(self, a, b):
        return a - b
"""

print("Processing sample code...\n")
processed_code = generate_docstring_scaffolds(sample_code)
```

```
print("--- Generated Source Code with Docstring Scaffolds ---")
print(processed_code)
```

output:

```
Processing sample code...
...

--- Generated Source Code with Docstring Scaffolds ---
def add(a, b):
    '''"""Placeholder docstring for add.

Args:
    *args: Description of arguments.

Returns:
    Description of the return value.
"""'''
    return a + b

class Calculator:
    '''"""Placeholder docstring for class Calculator."""'''

    def subtract(self, a, b):
        '''"""Placeholder docstring for subtract.

Args:
    *args: Description of arguments.

Returns:
    Description of the return value.
"""'''
        return a - b
```

Explanation:

The code creates a Calculator class that handles basic arithmetic operations and includes clear Google-style docstrings for documentation. Each method explains its purpose, inputs, outputs, and possible errors, making the code easier to understand, maintain, and reuse.